

Hamming Enhanced RISC Module

Nayan Tiwari

*Electronics and Communication Engineering
Jaypee Institute of Information Technology*

Noida, India

naayan2005@gmail.com

Rajveer Taneja

*Electronics and Communication Engineering
Jaypee Institute of Information Technology*

Noida, India

rajveertaneja01@gmail.com

Yashaswini Mahanta

*Electronics and Communication Engineering
Jaypee Institute of Information Technology*

Noida, India

mahantayashaswini@gmail.com

Abstract—Often in data transmission, data bits get flipped due to external disturbances such as power surges, radio frequency interference and electromagnetic radiation. This is known as Single Event Upset (SEU). This results in data corruption and can lead to failure of sensitive microcontroller-based systems. Multiple error detection and correction methods have been implemented in the past, but they often suffer from high hardware overhead or are too heavy for more primitive RISC-based microcontrollers. This paper aims to solve the problem of SEU in RISC-based microcontrollers while also tackling the issues of high hardware overhead, unhardened voters and time and space complex software methods. We find it is faster than pre-existing software methods and is useful in optimizing data processing in microcontrollers.

Index Terms—Hamming code, Forward Error Correction (FEC), RISC architecture

I. INTRODUCTION

In microcontrollers and microprocessors, where electronic components and circuits are at a micron level [1], external factors such as electromagnetic radiation and high energy particles interact and cause disturbances and bit flips in the data being received/transmitted. This disturbance is known as Single Event Upset (SEU) [2]. This often leads to system failures and is an even bigger concern when it comes to microcontroller-based space systems which transmit/receive sensitive information and are more so affected by high energy particles in space [3]. Hence, it becomes critical to not only eliminate such data corruption, but also do so in a refined manner.

This paper aims to focus on a hardware approach to detect and correct SEUs. It does so by implementing a Hamming Code [4] based module, placed between the microcontroller and the RF module which detects and corrects data before sending it to the microcontroller for processing or before sending it for transmission via the RF module.

A. Acronyms

RISC: Reduced Instruction Set Computer, FEC: Forward Error Correction SEU: Single Event Upset, HERM: Hamming

Enhanced RISC Module, RF: Radio Frequency, TCL: Tool Command Language

II. METHODOLOGY

A. Research Approach (The Solution)

The goal of this research is to replace traditional software based error correction with a dedicated hardware unit. Traditionally, microcontrollers handle error detection and correction by coding in C. This increases latency and puts a heavy load on the microcontroller. Furthermore, if the software detects an error it cannot fix, it must request the entire data packet to be sent again, which wastes time and power.

Our solution is the **HERM (Hamming Enhanced RISC Module)**. This is a hardware level module that sits between the Microcontroller (MC) and the RF module. It acts as a bridge that encodes data on the transmitter side and decodes it on the receiver side, fixing errors before it ever reaches the microcontroller.

B. Steps Followed

- Designed the 12,8 Hamming logic to generate 4 parity bits for 8 bits of data.
- Modeled the hardware as a combinational circuit to eliminate CPU cycle consumption.
- Mapped parity bits to power of two positions (1, 2, 4, 8).
- Verified the design via fault injection simulations to test self-correction capabilities.

C. Tools Used

- **EDA Tools:** Xilinx Vivado Design Suite (Synthesis, Implementation, and Simulation).
- **Scripting & Automation:** Tcl (Tool Command Language) for simulation control and running testbench sequences.
- **Verification:** Verilog Testbenches and Vivado Waveform Viewer.

D. Algorithm and Mathematics

The module implements the Hamming(12,8) Algorithm. Parity bits (P) are generated using parallel XOR operations to ensure minimal propagation delay:

$$P_1 = D_0 \oplus D_1 \oplus D_3 \oplus D_4 \oplus D_6 \quad (1)$$

$$P_2 = D_0 \oplus D_2 \oplus D_3 \oplus D_5 \oplus D_6 \quad (2)$$

$$P_4 = D_1 \oplus D_2 \oplus D_3 \oplus D_7 \quad (3)$$

$$P_8 = D_4 \oplus D_5 \oplus D_6 \oplus D_7 \quad (4)$$

The **Syndrome** (S) is computed by comparing received parity bits with recalculated values. The non-zero integer result of S points directly to the bit position requiring correction. While optimized software implementations such as those utilizing lookup tables achieve approximately 45-50 cycles [5], they remain sequential. HERM utilizes parallel hardware logic to achieve the same result in a single clock cycle.

E. System Features

- **Transparency:** The module is transparent to the microcontroller, requiring no software changes to existing firmware.
- **Deterministic Latency:** The hardware ensures a fixed, near zero propagation delay, providing predictable timing for RF communication.

III. RESULTS

The HERM was simulated using verilog simulation in Xilinx Vivado. We wrote code for encoder, which successfully generated parity bits and produced a 12-bit Hamming(12,8) codeword for the given 8-bit input data before sending it to RF module. The decoder processed the received codeword and produced the corresponding 8-bit output data. Fault injection was done during testbenching. Different noise channels were made to ensure that our module is able to fix errors. Three verification scenarios were tested:

- 1) **No Noise Condition:** In this condition, no bit corruption was introduced in the input. The decoded output exactly matched the transmitted input data. The test status was recorded as 'PASS(Data Intact)'.
- 2) **Single Data Bit Flip:** In this condition, a single bit in the encoded input data was inverted during transmission. The decoder detected a non-zero syndrome value and corrected the error. The restored output matched the original input data, and the test status was recorded as 'SUCCESS(Error Corrected)'.
- 3) **Parity Bit Flip:** In this condition, a parity bit (P_4) was intentionally flipped. The decoder detected the parity error and managed it correctly. The output data received remained correct, and the test status was recorded as 'SUCCESS(Parity Error Handled)'.

Waveform and TCL console analysis confirmed that the encoder output, noisy channel input, and decoded output behaved as expected across all test cases. In each scenario,

the corrected output data matched the original input within a single clock cycle after decoding.

```
=====
HERM SYSTEM VERIFICATION:
=====
[TIME: 40000] TEST 1: NO NOISE
TX Data: 41 | Codeword: 010010000100 | RX Output: 41
STATUS: PASS (Data Intact)

[TIME: 120000] TEST 2: DATA BIT FLIP (Bit 6)
TX Data: aa | Original Codeword: 101001011000
           | Noisy Codeword: 101001111000
           | Syndrome: 0010
RX Output: aa
STATUS: SUCCESS (Error Corrected)

[TIME: 200000] TEST 3: PARITY BIT FLIP (P4)
RX Output: f0
STATUS: SUCCESS (Parity Error Handled)

=====
VERIFICATION COMPLETE
=====
```



Fig. 1. Simulation results: (left) TCL console output, (right) waveform verification under fault injection.

IV. CONCLUSION

In this work a hardware module was designed that can generate parity bits to encode the data and decode data after fixing any errors or SEUs. It can replace the existing software based error correction models [5] that take time and comparatively more cycles to do the same work. We verified the results with testbenching and graphs which produced the desired results.

It is also an easier and low cost approach in lightweight RISC based architectures [6] when compared to the HARV [7] processor, which suffers from high hardware overhead and has unhardened voters that remain susceptible to errors.

REFERENCES

- [1] Page name: Single-event upset, Author: Wikipedia contributors, Publisher: Wikipedia, The Free Encyclopedia, Date of last revision: 9 December 2025 16:55 UTC
- [2] P. E. Dodd and L. W. Massengill, "Basic mechanisms and modeling of single-event upset in digital microelectronics," in *IEEE Transactions on Nuclear Science*, vol. 50, no. 3, pp. 583-602, June 2003
- [3] A. K. Singh, "Error detection and correction by hamming code," *2016 International Conference on Global Trends in Signal Processing, Information Computing and Communication (ICGTSPICC)*, Jalgaon, India, 2016, pp. 35-37
- [4] T.Manivannan1, Y.BasheerBaba2, B.Anish Kumar3, and D.Narasimha Siva4, "Advanced VLSI technique for error detection and correction in space systems," *Orange Publications International Journal for Interdisciplinary Sciences and Engineering Applications IJISEA - An International Peer- Reviewed Journal*, 2024, Volume 5 Issue 2, ISSN: 2582 - 6379
- [5] David Cook, Hamming Error Correcting Code (ECC), Available: <https://www.robotroom.com/index.html>
- [6] Aswin Sreekumar, Bolupadra Sai Shankar, B. Naresh Kumar Reddy, Integrating error correction and detection techniques in RISC-V processor microarchitecture for enhanced reliability, *Integration*, Volume 100, 2025, 102282, ISSN 0167-9260,
- [7] Douglas Almeida dos Santos. Development of a hardened processor for harsh environment applications on RISC-V architecture.. Micro and nanotechnologies/Microelectronics. Université de Montpellier, 2023. English. (NNT : 2023UMONS089). (tel-04612443), Available: <https://theses.hal.science/tel-04612443/>